AIL 7002 : Assignment 1

Siddharth (2025AIB2670)

Meghender Pal (2025AIB2563)

CRICKET TOURNAMENT SCHEDULING USING GRAPH COLORING

Converting the problem to a graph coloring problem

We want to find a schedule for our tournament involving n teams where each team plays the other home and away, i.e., for any two teams X and Y, they should play each other twice. Once at team X's home ground and once at Y's home ground. We have been given a constraint that each team can play only one game every 2 days.

Total number of matches to be played for n teams -> $2\ {}^{n}C_{2} = 2 \cdot n!/ (2! (n-2)!)$

$= 2. n (n-1) (n-2)!/ (2 \cdot (n-2)!)$

$= n(n-1)$

Hard Constraint: For validity. They specify our solution space. For the given problem, we have a hard constraint that –
- A team can't play more than one match on a given day.
- A team can't play on consecutive days.

Soft Constraint: For optimality. They specify what a good solution and a bad solution are in our solution space. For the given problem, the soft constraint will be to minimize the number of days over which the tournament is conducted.

This problem can be converted into a graph coloring problem. Our graph will be a collection of nodes and edges such that ->

- **Node:** Each node represents a game to be played. In our graph, we will have a total of n(n-1) nodes. A node XY means a match between team X and team Y at X's home ground.
- **Edges:** Two nodes are connected by an edge if they correspond to two games which cannot be scheduled together (either on the same day or on consecutive days. Two games cannot be scheduled together if they have a team in common. Therefore, a node

XY (corresponding to a match between X and Y) will be connected to all the nodes having X or Y as one of the teams.

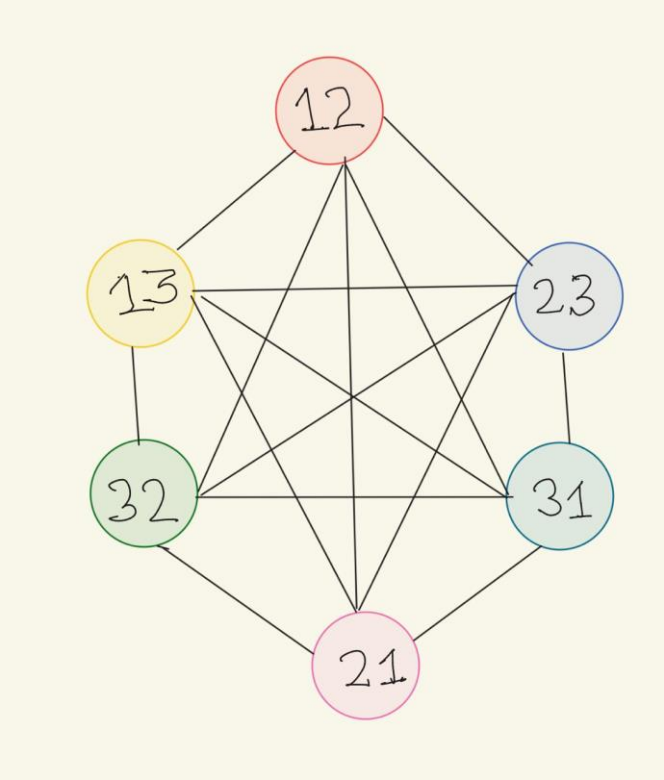
- **Graph Coloring:** Now, this is a graph coloring problem. In a valid coloring of the graph, each color will represent games that can be scheduled together.

This problem seems a bit different from the usual graph coloring we are used to. Here, we not only have to check that 2 nodes/matches that have an edge between them can't be scheduled on the same day, but also that they can't be scheduled on consecutive days. Hence, two connected nodes can't belong to the same subset, but they can't belong to adjacent subsets either.

But if we do some reasoning, the problem comes back to simple graph coloring with all the additional complexity removed. Assume d is a playing day and no games were played on day (d-1). Our greedy algorithm ensures that each set tries to maximize the number of nodes it takes in. If the number of teams n is even, n/2 games are conducted on day d, and no team is available to play on day d+1. If the number of teams n is odd, (n-1)/2 games are conducted on day d, and only 1 team is available to play on day d+1. Either way, no games will be conducted on day d+1.

Therefore, in a valid coloring of a graph, each color will give us a set of games that can be conducted in a single 2-day window. Within each overlapping window, the games can always be kept on the first day to avoid any overlapping between successive windows. Between every two game days, we will have an empty day on which no games are played.

A simple example for small n -> Let's choose n=3 and call our teams as the number 1,2,3. We will have a total of n(n-1) i.e. 3(3-1)=6 nodes in our graph. Since, in 3 teams, we will always have 2 teams playing a game, all the nodes will have an edge with each other. Also, on any given day, we can have a maximum of one game. So, our graph for this problem will be a 6-clique. A total of 6(5)/2= 15 edges will be present. As shown in the figure down here.



(A node 12 indicates a match between team 1 and team 2 where team 1 is the home team and team 2 is the away team.)

This will be the schedule generated by our greedy graph coloring algorithm-
(1 2)

(1 3)

(2 1)

(2 3)

(3 1)

(3 2)

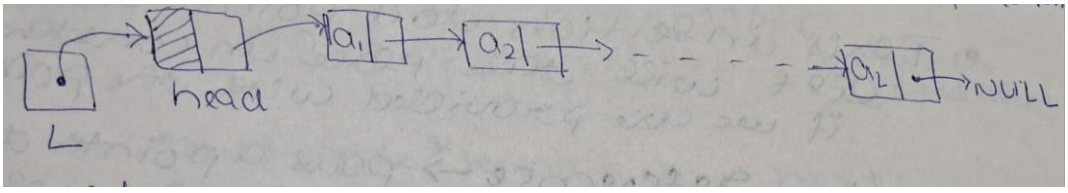
Implementing ADTs (Abstract Data Types)

Following ADT's will be required to implement a greedy graph coloring algorithm to solve this problem –

- **List:** A list is a set of nodes assigned a given color. Each list/color corresponds to the set of games that can be conducted on the first day of a two-day window.
- **Graph:** A collection of nodes and edges. For simpler problems, we represented each node of a graph by an integer. But in the given problem, each node has some sort of internal representation of its own. Therefore, it will also be a user-defined datatype.
  No. of nodes in a graph for n teams -> 2 ${}^{n}C_{2} = n(n-1)$
  No. of edges in a graph for n teams -> (no. of nodes)*(degree of each node)/2
  $= (n(n-1) * [1+ 2*2(n-2)])/2$

List ADT

1. **Mathematical Model ->**
   - Data: struct node{
     int data;
     node* next;
     };
     Nodes are the building blocks of our list. Each node consists of data, which is an integer, and next, which is a pointer to the next node. We have chosen a pointer-based implementation of a linked list.
   - Aggregation Mechanism: struct node* L;
     - A list is a pointer to a struct node. To differentiate between the List and an element of a list, we make our L point to a node head, whose data we don't worry about. The next of head will point to the first element of the list. The head can be thought of as the 0th element of our list. We'll assume 1-based indexing for our list nodes.
     - Each cell points to the next cell, forming a chain.



2. **Set of Operations ->**
    - MAKENULL(L): Creates an empty list. Make L point to the dummy head and make head point to null.

```
void MAKENULL(L)
          L ← new node
          L.next ← NULL
end
```

    - INSERT_FRONT (L, x): Insert a new node at the beginning of the list, right after the dummy head.

```
void INSERT_FRONT(L, x)
    new_node ← new node
    new_node.data ← x
    new_node.next ← L.next
    L.next ← new_node
end
```

    - INSERT_END (L, x): Insert a new node at the end of the list

```
void INSERT_END(L, x)
    new_node ← new node
    new_node.data ← x
    new_node.next ← NULL
    temp ← L
    while temp.next ≠ NULL do
        temp ← temp.next
    end while
    temp.next ← new_node
end
```

    - IS_EMPTY_LIST(L): Check if a list is empty. Will return True if a list has no data nodes and only has a head node.

```
bool IS_EMPTY_LIST(L)
    return (L.next = NULL)
end
```

    - GET_NODE (L, p): Return a pointer to the node at position p. Returns null if the node is out of range. Will be useful for a lot of later functions. We can store this pointer in a variable and make changes at that place directly without looping over the list again.

```
node* GET_NODE(L, index)
    curr ← L
    count ← 0
    while curr ≠ NULL and count < index do
```

```
        curr ← curr.next
        count ← count + 1
    end while
    return curr
end
```

- INSERT_AT (L, x, p): Insert a new node with data x at a particular position p in our list L. Makes use of GET_NODE (L, p-1).

```
void INSERT_AT(L, x, p)
    prev ← GET_NODE(L, p - 1)
    if prev = NULL then return
    new_node ← new node
    new_node.data ← x
    new_node.next ← prev.next
    prev.next ← new_node
end
```

- RETRIEVE (L, p): Retrieve the data of the element at position p in the list.

```
int RETRIEVE(L, p)
    curr ← GET_NODE(L, p)
    if curr = NULL then return -1
    return curr.data
end
```

- LOCATE (L, x): Returns the position of the first node having a data value of x in our list. If we can't find any such node return -1.

```
int LOCATE(L, x)
    curr ← L.next
    pos ← 1
    while curr ≠ NULL do
        if curr.data = x then return pos
        curr ← curr.next
        pos ← pos + 1
    end while
    return -1
end
```

- REMOVE_AT (L, p): Remove the node at position p in the list

```
void REMOVE_AT(L, p)
    prev ← GET_NODE(L, p - 1)
    if prev = NULL or prev.next = NULL then return
    to_delete ← prev.next
    prev.next ← to_delete.next
    delete to_delete
end
```

- REMOVE_FIRST (L, x): Remove the first node in the list having the data==x. Makes use of LOCATE (L, x).

```
bool REMOVE_FIRST(L, x)
    pos ← LOCATE(L, x)
    if pos = -1 then return false
    prev ← GET_NODE(L, pos - 1)
    if prev = NULL or prev.next = NULL
          then return false
    to_delete ← prev.next
    prev.next ← to_delete.next
    delete to_delete
    return true
end
```

- FIRST(L): Returns the pointer to the first element of list L.

```
node* FIRST(L)
    return L.next
end
```

- END(L): Returns the pointer to the last element of list L.

```
node* END(L)
    curr ← L
    while curr.next ≠ NULL do
        curr ← curr.next
    end while
    return curr
end
```

- NEXT (pp): Returns the pointer to the next node. You're given the pointer to the previous node pp.

```
node* NEXT(p)
    if p = NULL then return NULL
    return p.next
end
```

- PREVIOUS (L, pp): Returns a pointer to the previous node of the node pointed to by pp.

```
node* PREVIOUS(L, p)
    if L = NULL or p = L then return NULL
    curr ← L
    while curr ≠ NULL and curr.next ≠ p do
        curr ← curr.next
    end while
    return curr
end
```

Graph ADT

1. **Mathematical Model ->**
   - Data: struct Graph {
     int n;
     node** adj;
     };

     Each graph is of type struct graph. It has two elements. n of type integer which tells us the number of nodes in the graph and a variable adj which is a pointer to variable of type "pointer to struct node". The set of vertices is not stored in a list itself, but is implicitly defined by integer indices. The vertices are the integers 0,1,2,…,n−1.
   - Aggregation Mechanism: node** adj is the primary aggregation mechanism. It's a list of size n. It holds the pointer to n list heads. Each of the list heads is the 0th node of the adjacency list of the corresponding vertex. An edge (u,v) exists in the graph if and only if vertex v is in the adjacency list of u, and vertex u is in the adjacency list of v.
2. **Set of Operations ->**
   - CONSTRUCT_GRAPH (G, n): Construct a graph with n vertices and no edges.

```
void CONSTRUCT_GRAPH(G, n)
    G.n ← n
    allocate list G.adj of size n
    for i ← 0 to n-1 do
        MAKENULL(G.adj[i])
    end for
end
```

   - ADD_EDGE (G, u, v): Add an undirected edge between vertices u and v. For this, we add v and u to the adjacency list of each other.

```
void ADD_EDGE(G, u, v)
    INSERT_END(G.adj[u], v)
    INSERT_END(G.adj[v], u)
end
```

   - REMOVE_EDGE (G, u, v): Remove the undirected edge (u,v) if present. For this we remove both the vertives v and u from the adjacency list of each other.

```
void REMOVE_EDGE(G, u, v)
    REMOVE_FIRST(G.adj[u], v)
    REMOVE_FIRST(G.adj[v], u)
end
```

- GET_NEIGHBORS (G, u): Returns the adjacency list for vertex v. We can iterate over this list, using the functions like first and next.

```
node* GET_NEIGHBOURS(G, u)
    return G.adj[u]
end
```

- DEGREE (G, u): Return the degree (number of neighbours) of vertex u.

```
function DEGREE(G, u) returns integer
    d ← 0
    curr ← FIRST(G.adj[u])
    while curr ≠ NULL do
        d ← d + 1
        curr ← NEXT(curr)
    end while
    return d
end
```

- REMOVE_NODE (G, u): Remove a vertex u and all incident edges.

```
procedure REMOVE_NODE(G, u)
    if u < 0 or u ≥ G.n then return

    for v ← 0 to G.n - 1 do
        if v ≠ u then
            REMOVE_FIRST(G.adj[v], u)
        end if
    end for

    curr ← FIRST(G.adj[u])
    while curr ≠ NULL do
        nxt ← NEXT(curr)
        delete curr
        curr ← nxt
    end while

    G.adj[u].next ← NULL
end
```